Presentation 1

STAT 390 | Winter 2026 | Team 6

Northwestern

Data Pre-Processing (Programming)

Henry Chau

main.prepare_data()

- In main.py, prepare_data() is a helper function called by main()
- Calls various functions imported from data_utils.py, utils.py, trainer.py
- Load the expert's classifications but drop case 91
- Reclass 1 → 0 (benign), 3 & 4 → 1 (high-grade), dropping low-grade observations
- Case ids split into training, validation, testing sets at 60-20-20 ratio, stratifying twice on C-MIL classification

main.prepare_data() (cont.)

```
Label distribution per split:
label   0   1
split
test    6  12
train  17  33
val     5  12

Mean patches per stain per split:
       h&e_patches  melan_patches  sox10_patches
split
test         409.2          250.9          238.1
train        470.5          263.2          219.9
val          504.8          241.9          172.8

Median patches per stain per split:
       h&e_patches  melan_patches  sox10_patches
split
test         364.5          173.5          160.0
train        324.0          194.0          182.0
val          273.0          175.0          162.0

Missing stain proportion:
       h&e_missing  melan_missing  sox10_missing
split
test           0.0          0.111          0.000
train          0.0          0.000          0.020
val            0.0          0.059          0.059

Total cases: 85
Total patches: 79,476
```

```
Patch count by class for Train:
  Benign (0):      15639 patches
  High-grade (1):  32040 patches

Patch count by class for Validation:
  Benign (0):      5968 patches
  High-grade (1):  9663 patches

Patch count by class for Test:
  Benign (0):      4130 patches
  High-grade (1):  12036 patches

Data Integrity Check for Train:
Stain coverage:
  h&e: 50/50 (100.0%)
  melan: 50/50 (100.0%)
  sox10: 49/50 (98.0%)
No issues found

Data Integrity Check for Validation:
Stain coverage:
  h&e: 17/17 (100.0%)
  melan: 16/17 (94.1%)
  sox10: 16/17 (94.1%)
No issues found

Data Integrity Check for Test:
Stain coverage:
  h&e: 18/18 (100.0%)
  melan: 16/18 (88.9%)
  sox10: 18/18 (100.0%)
No issues found
```


note: these outputs are from ‘training_logs_7341637.log’ on Quest

Comments

- The current code works but could use some tidying up
- Some functions like data_utils.check_disjoint_sets() have unused parameters
- trainer.count_patches_by_class(), is unrelated to the training operations of its file. Suggesting move to data_utils.py
- utils.print_model_summary() incorrectly prints the sum of total elements in the input tensors instead of the claimed number of parameters
- Initial organization of data could be made more intuitive by building out from cases>stain>slice>patches

Code Organization

- The code ran for a little over 11 hours on QUEST using 6/7 cores
- Memory usage indicates efficient code and we can consider adjusting memory allocation

```
[trd1531@quser44 e32998]$ seff 7341637
Job ID: 7341637
Cluster: quest
User/Group: trd1531/trd1531
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 16
CPU Utilized: 6-15:57:38
CPU Efficiency: 88.77% of 7-12:12:16 core-walltime
Job Wall-clock time: 11:15:46
Memory Utilized: 20.26 GB
Memory Efficiency: 15.83% of 128.00 GB (128.00 GB/node)
```

Training & Testing (Programming)

Vicky Xu

Code Organization

Overview

Step 1: Configuration and Setup

Step 2: Data Pre-processing

Step 3: Model Architecture

Step 4: Training and Prediction

Step 5: Result Analysis

Code Organization

Step 1: Configuration and Setup

`config.py`: Centralizes all hyperparameters, data paths, and model settings.

`main.py`: Controls the overall pipeline by parsing command-line arguments, setting random seeds for reproducibility (via utils.py), and selecting the execution device (CPU or GPU).

Code Organization

Step 2: Data Pre-processing

`data_utils.py`

1. Label Loading: reads case_grade_match.csv to map case IDs to class labels.
2. Patch Grouping: The script scans the patch directory and uses regex patterns to group PNG files into slices and cases based on filenames.
3. Stratified Splitting: `split_by_case_stratified` divides cases into train/validation/test sets while preventing data leakage (no case appears in multiple splits).

`dataset.py`

4. Dataset Creation: `StainBagCaseDataset` uses lazy loading, loading and transforming images on demand during training instead of preloading them into memory.

Code Organization

Step 3: Model Architecture

`models.py`: Implements a three-level hierarchical attention architecture:

1. Patch-Level: A frozen DenseNet121 extracts patch features, which are aggregated into a slice embedding via attention pooling.

2. Stain-Level: Slice embeddings from the same stain are combined into a stain embedding.

3. Case-Level: Stain embeddings (H&E, Melan, SOX10) are weighted by attention to produce the final case-level prediction.

Code Organization

Step 4: Training and Prediction

`trainer.py`:

1. `MILTrainer`: Handles training, validation, and early stopping.
2. Training Flow: For each batch (typically one case), it runs a forward pass through the hierarchical model, computes weighted `CrossEntropyLoss` to address class imbalance, and backpropagates to update the attention layers.

Step 5: Result Analysis

`attention_analysis.py`: Extracts learned attention weights during evaluation or after training.

Output: Generates visualizations (rank plots and patch images) highlighting the slices and patches that most influenced the model's predictions.

Code Efficiency and Redundancy

Efficiency

1. Frozen Feature Extractor: Freezing the DenseNet121 backbone is highly efficient, as it avoids the massive computational cost of training a deep CNN from scratch.

2. Lazy Loading: Loading images in dataset.py prevents out-of-memory errors that would occur if trying to load thousands of high-resolution patches at once.

Code Efficiency and Redundancy

Redundancy

1. Redundant Pre-processing: Images are resized and normalized every epoch. Precomputing and saving CNN features (e.g., .pt or .npy) would enable training on embeddings instead of raw images.

2. Batch Size = 1: While common in MIL, single-case batches underutilize GPU parallelism. A custom collation strategy for multi-case batching could improve throughput.

3. Manual Normalization in Analysis: `attention_analysis.py` duplicates denormalization logic from `dataset.py`, creating maintenance risk if normalization settings in `config.py` change. Centralizing this logic would ensure consistency.

Mathematical Operations

Abhi Vinnakota

Tensor Transformations/Feature Extraction

Data Preprocessing

$$\text{Training: } \mathcal{T}_{\text{train}} = \text{Normalize}(\text{Augment}(I)) \to \mathbb{R}^{3\times224\times224}$$

$$\text{Val/Test: } \mathcal{T}_{\text{val}} = \text{Normalize}(\text{Resize}(I)) \to \mathbb{R}^{3\times224\times224}$$

Feature Extraction

$$\mathbf{x}_p \xrightarrow{\text{DenseNet121 (frozen)}} \mathbb{R}^{1024\times7\times7} \xrightarrow{\text{Pool}} \mathbb{R}^{4096}$$

$$\xrightarrow{\text{Project + ReLU + Dropout}} \mathbf{e}_p \in \mathbb{R}^{512}$$

$$\mathbf{e}_p = \text{Dropout}_{0.3}(\text{ReLU}(\mathbf{W}_{\text{proj}}\mathbf{h}_p + \mathbf{b}_{\text{proj}}))$$

Hierarchical Attention Mechanism

General Attention Pooling

Given embeddings $\mathbf{E} \in \mathbb{R}^{M \times D}$:

$$\mathbf{a} = \mathbf{W}_2 \tanh(\mathbf{W}_1 \mathbf{E}^\top + \mathbf{b}_1) \in \mathbb{R}^M$$

$$\boldsymbol{\alpha} = \text{softmax}(\mathbf{a}) = \frac{\exp(\mathbf{a})}{\sum_{j=1}^{M} \exp(a_j)}$$

$$\mathbf{z} = \mathbf{E}^\top \boldsymbol{\alpha} \in \mathbb{R}^D$$

Hierarchical Levels

$$\mathbf{W}_1 \in \mathbb{R}^{128 \times 512}, \mathbf{W}_2 \in \mathbb{R}^{1 \times 128}$$

Patch → Slice: $\mathbf{z}_{\text{slice}}^{(s,t)} = \sum_{p=1}^{P} \alpha_p^{(s,t)} \mathbf{e}_p^{(s,t)} \in \mathbb{R}^{512}$

Slice → Stain: $\mathbf{z}_{\text{stain}}^{(t)} = \sum_{s=1}^{S_t} \beta_s^{(t)} \mathbf{z}_{\text{slice}}^{(s,t)} \in \mathbb{R}^{512}$

Stain → Case: $\mathbf{z}_{\text{case}} = \sum_{t=1}^{3} \gamma_t \mathbf{z}_{\text{stain}}^{(t)} \in \mathbb{R}^{512}$

Complete Tensor Transformation

Stage	Shape
Raw Patch	$\mathbb{R}^{3\times224\times224}$
After DenseNet121 (frozen)	$\mathbb{R}^{1024\times7\times7}$
After Adaptive Pool	$\mathbb{R}^{4096}$
Patch Embedding	$\mathbb{R}^{512}$
Level 1: Patch Attention	
Input (per slice)	$\mathbb{R}^{P\times512}$ (cap $P = 800$)
Slice Embedding	$\mathbb{R}^{512}$
Level 2: Stain Attention	
Input (per stain)	$\mathbb{R}^{S\times512}$
Stain Embedding	$\mathbb{R}^{512}$
Level 3: Case Attention	
Input (per case)	$\mathbb{R}^{3\times512}$ (H&E, Melan, SOX10)
Case Embedding	$\mathbb{R}^{512}$
Classification Logits	$\mathbb{R}^{2}$ (benign, high-grade)

Final Classification: $\mathbf{y}_{\text{logits}} = \mathbf{W}_{\text{clf}}\text{Dropout}_{0.3}(\mathbf{z}_{\text{case}}) + \mathbf{b}_{\text{clf}}$

Parameters and Hyperparameters

Pretrained:

DenseNet121: $\sim$7M params

Trainable Params

Patch Projector: $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{512\times4096} \rightarrow$ 2.1M params

Patch Attention: $\mathbf{W}_1 \in \mathbb{R}^{128\times512}$, $\mathbf{W}_2 \in \mathbb{R}^{1\times128} \rightarrow$ 66K params

Stain Attention: Same structure $\rightarrow$ 66K params

Case Attention: Same structure $\rightarrow$ 66K params

Classifier: $\mathbf{W}_{\text{clf}} \in \mathbb{R}^{2\times512} \rightarrow$ 1K params

Hyperparameters

Architecture: $D = 512$, $H = 128$, $P_{\text{max}} = 800$, $p_{\text{drop}} = 0.3$

Training: $\eta = 3 \times 10^{-4}$, $\lambda = 2 \times 10^{-4}$, $B = 1$, $E_{\text{max}} = 30$

Class weights: $[w_0, w_1] = [2.5, 1.0]$

Scheduler: ReduceLROnPlateau (patience=3, factor=0.3)

Loss Term

Weighted Cross Entropy

For case with true label $y \in \{0, 1\}$ and logits $\mathbf{y}_{\text{logits}} = [y_0, y_1]$:

$$\mathcal{L}_{\text{CE}}(y, \mathbf{y}_{\text{logits}}) = -w_y \log\left(\frac{\exp(y_y)}{\sum_{k=0}^{1} \exp(y_k)}\right)$$

where $w_0 = 2.5$ (benign), $w_1 = 1.0$ (high-grade)

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{reg}}$$

L2 Regularization (Weight Decay)

$$\mathcal{L}_{\text{reg}} = \frac{\lambda}{2} \sum_{\theta \in \Theta_{\text{trainable}}} \|\theta\|_2^2 \quad (\lambda = 2 \times 10^{-4})$$

Forward/Backprop

Forward Pass:

for each stain $t \in \{\text{H\&E, Melan, SOX10}\}$ **do**
  for each slice s in stain t **do**
    for each patch p (up to 800) **do**
      $\mathbf{e}_p \leftarrow \text{Project}(\text{Pool}(\text{DenseNet}(\mathbf{x}_p)))$ // frozen CNN
    end for
    $\mathbf{z}_{\text{slice}}^{(s,t)} \leftarrow \text{AttentionPool}(\{\mathbf{e}_p\})$
  end for
  $\mathbf{z}_{\text{stain}}^{(t)} \leftarrow \text{AttentionPool}(\{\mathbf{z}_{\text{slice}}\})$
end for
$\mathbf{z}_{\text{case}} \leftarrow \text{AttentionPool}(\{\mathbf{z}_{\text{stain}}\})$
$\mathbf{y}_{\text{logits}} \leftarrow \mathbf{W}_{\text{clf}}\text{Dropout}(\mathbf{z}_{\text{case}}) + \mathbf{b}_{\text{clf}}$

Backward Pass (Key Gradient):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_{\text{stain}}^{(t)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{\text{case}}} \cdot \gamma_t \quad \text{(attention weights control gradient flow)}$$

Adam Optimization

Training Loop:

1: Initialize Θ randomly, load frozen DenseNet121
2: **for** epoch $= 1$ to 30 **do**
3: **for** each case c **do**
4: $\mathbf{y}_{\text{logits}} \leftarrow \text{ForwardPass}(c, \Theta)$
5: $\mathcal{L} \leftarrow \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{reg}}$
6: $\Theta \leftarrow \text{AdamUpdate}(\Theta, \nabla_{\Theta}\mathcal{L})$
7: **end for**
8: **if** no val improvement for 5 epochs **then**

Adam Update Rule: At iteration t, for parameter θ:

$$g_t = \nabla_\theta \mathcal{L}_{\text{total}}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Hyperparameters: $\eta = 3 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Testing & Training (Programming)

Zijin Zeng

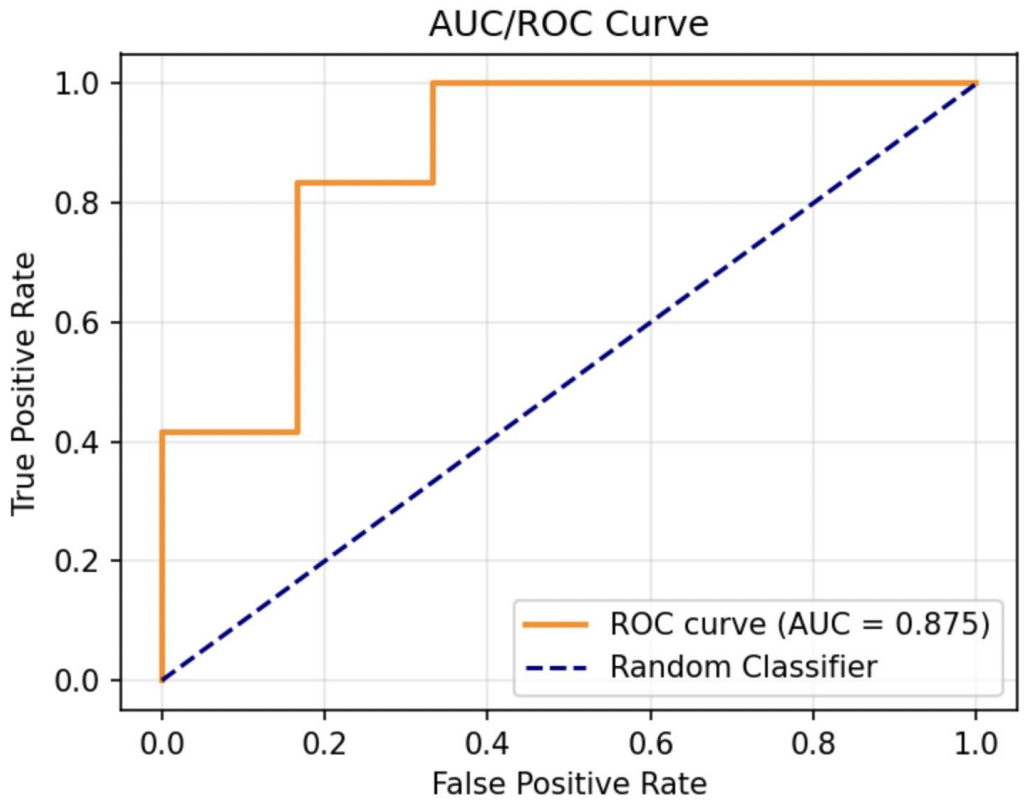
Summary

87.5%

Test Set AUC

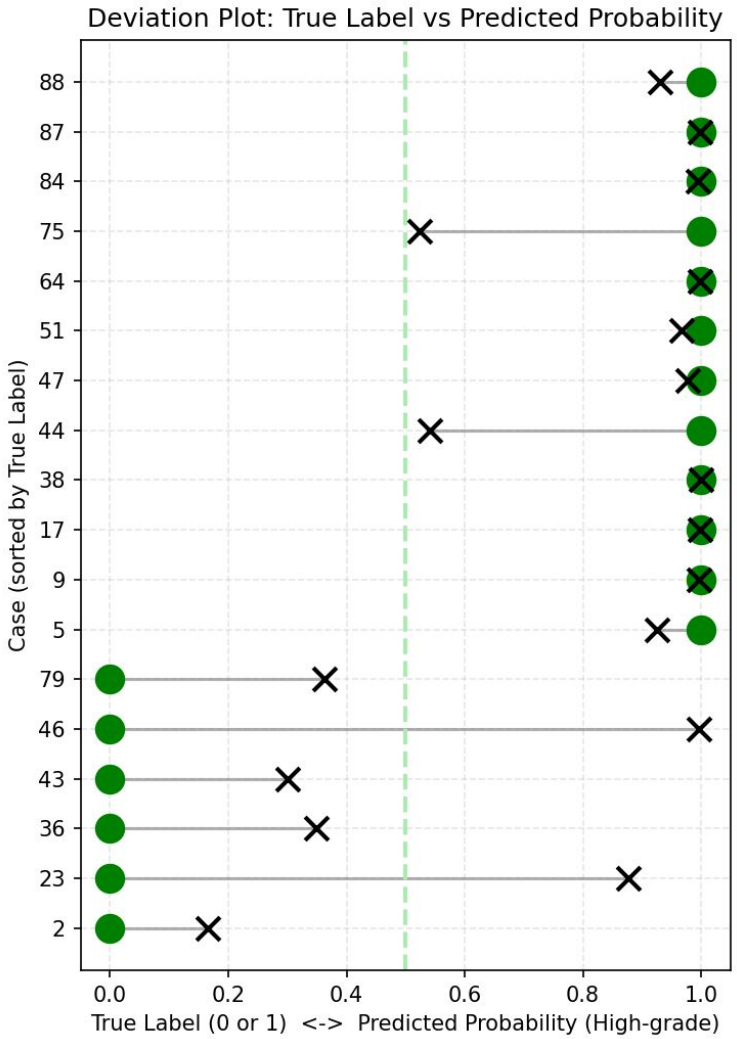
39%

Expert Pathologists Accuracy



The MIL framework effectively identifies High-grade cases, showing a much clearer separation than pathologist consensus.

Deviation Plot

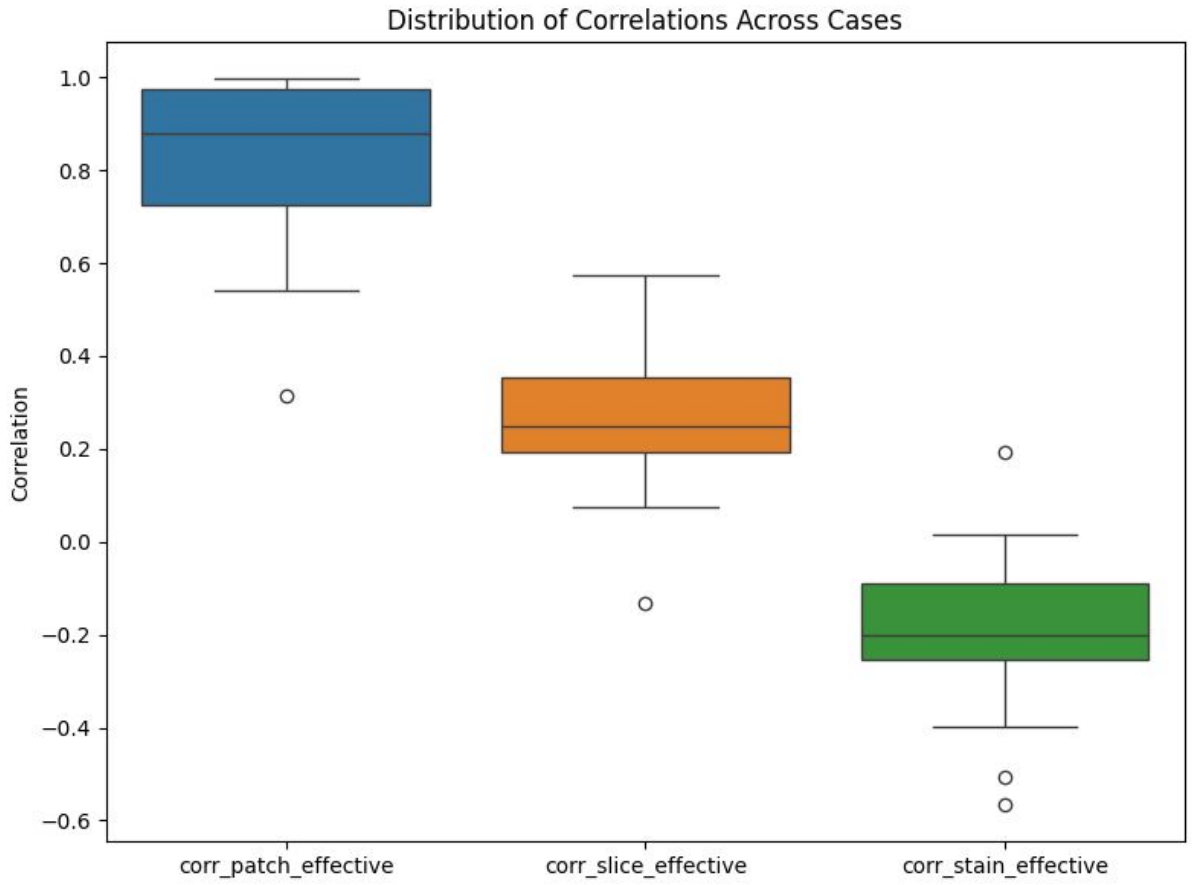


- Green dots show the **true label** (0=benign, 1=high-grade); Black Xs show the model's **predicted P.**
- The gray segment is the prediction gap.
- The dashed line at 0.5 is the decision threshold.

- **The model predicts most cases accurately.**
- Performance is stronger for **high-grade** cases. Most remaining errors are **benign false positives** (benign cases assigned high P).

- Tune the operating threshold on the validation set
- Give extra training emphasis to benign with high abs_gap so the model learns the boundary cues and cuts false alarms
- Add a class-balance, if skewed, use class-weighted loss to avoid a majority-class bias.

Distribution of Correlation

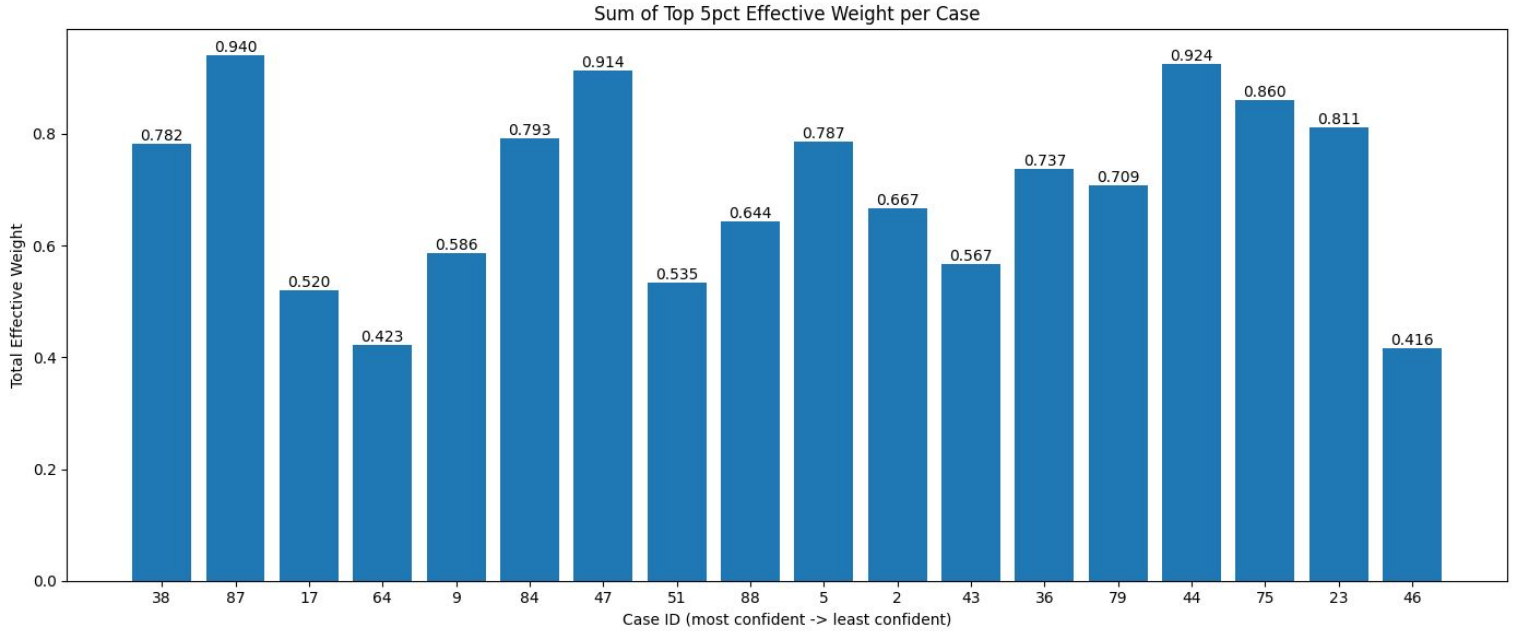


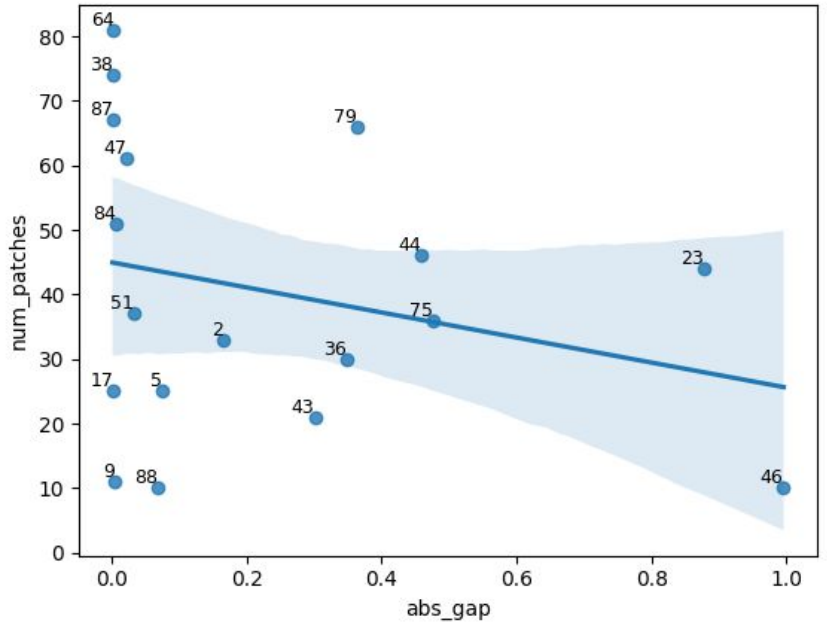
Each box shows, across cases, how strongly an attention level tracks the final patch contribution

The model's decisions are driven primarily by patch-level evidence, while slice and stain-level signals are less aligned with effective evidence and may require more robust fusion.

- Strengthen patch attention
- Use stain information more conservatively: trust stain signals more when different stains agree, and fall back to patch-driven decisions when they conflict.

Patch Count / Evidence Reliance





- Error (abs_gap) shows no clear dependence on the number of available patches
- Evidence concentration (top-5% effective weight sum) also does not change monotonically with error
- **Rather than extracting more patches, the priority is improving patch quality and coverage.** We can verify this by filtering out low-information patches (e.g., mostly background, blurred) and checking that high-error cases are not simply those with fewer patches.

Reference

- Lecture: STAT 390: Data science project CMIL Classification Problem statement & Data
- Visualization_Prediction — Fall 2025 MIL Trainer Final Model and Analysis
- 2025 Fall Presentation Slides
- Jack Yu’s presentation going over the outline of the code from last quarter